



UNIVERSITÀ DI PARMA

I Linguaggi di Programmazione

*There will always be things we wish to say in
our programs that in all languages can only
be said poorly*

Alan Perlis

- Piccolo riepilogo
- Il linguaggio macchina
- Assembly
- Linguaggi di Alto Livello
- Linguaggi Interpretati e Compilati

- Dove si trova il programma da eseguire?
 - In memoria → “numeri”
- La control unit esegue i programmi ovvero:
 - Legge da memoria la relativa istruzione da eseguire → **Fetch**
 - Decodifica il suo significato ovvero cosa deve fare → **Decode**
 - Esegue quanto richiesto pilotando gli altri sistemi → **Execute**
 - Scrive eventuali risultati elaborazione → **Store**
 - Ripete...

- Come è fatta una istruzione decodificata dall'unità di controllo?
- Tipicamente più “parti”
 - Operation code → mi dice che istruzione è
 - Operando/i → mi fa capire su cosa si opera
- Instruction Set → dipende da processore!
- Esempio: ADD X, Y
 - Somma il contenuto della memoria all'indirizzo X a quello in Y e scrivi il risultato in X

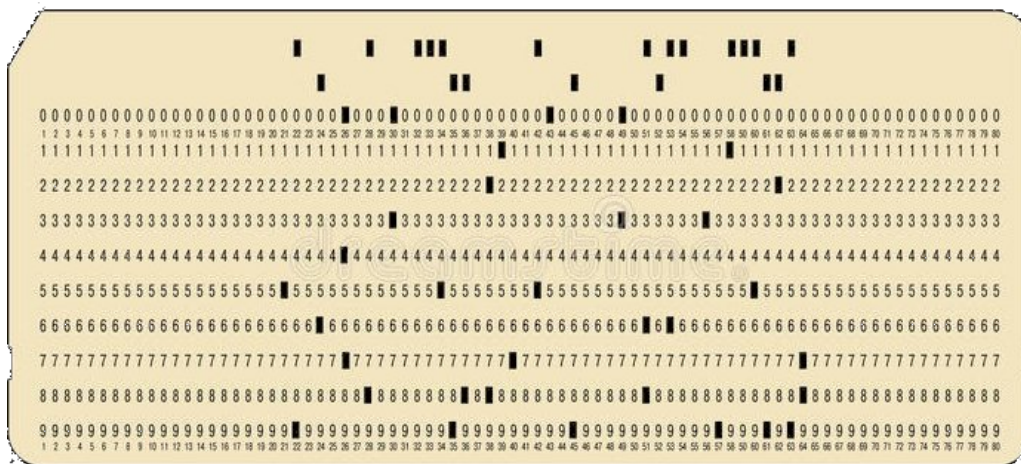
Opcode (8 bit)	Indirizzo X (16 bit)	Indirizzo Y (16 bit)
00001001	0000000001100011	0000000001100100

- Codice numerico che corrisponde alle possibili istruzioni eseguibili da una CPU
- Diverse per ogni tipo di processore
 - Instruction Set
- Ad esempio l'Instruction set dei processori ARM è completamente differente da quello dei processori Intel x86
 - Differenti modelli della stessa famiglia di processori presentano Instruction Set con variazioni

6502 Machine Language Hex Codes

[illegible]

- Non mnemonico → difficoltà di utilizzo
- Non portabile
 - Codice sviluppato per una famiglia di CPU non utilizzabile altrove
- All'atto pratico non usato direttamente

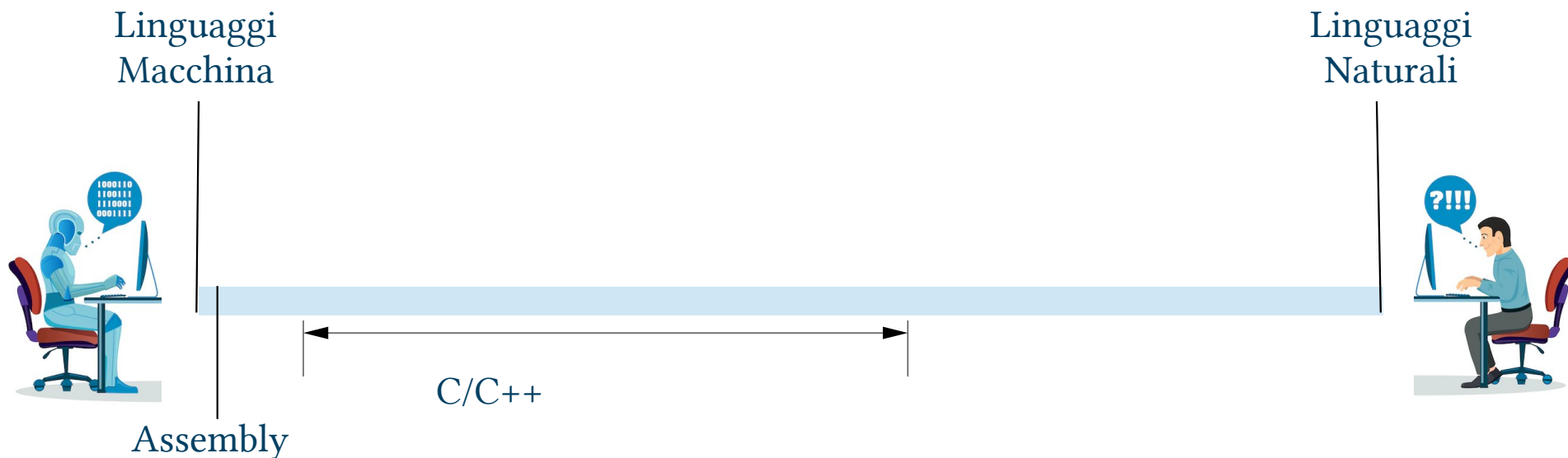


- Sostituzione codice numerico con simboli mnemonici
- ~1:1

```
START: MOV    AX, BX
        CMP    AX, 12h
        JZ     EQUAL
        INT    21h
        RET
EQUAL:  MOV    BL, 82h
```


- Molto più facile del codice macchina!
 - Sia da scrivere che da leggere
- Ma devo comunque ottenere il codice macchina
- Assembler: programma che legge le istruzioni assembly e le traduce in istruzioni macchina
 - Quasi traduzione 1 a 1
- Comunque linguaggio complesso
 - Molte istruzioni anche per operazioni semplici
- Non portabile

- Non più corrispondenza 1:1 con istruzioni linguaggio macchina
 - 1 istruzione può corrispondere a decine di istruzioni linguaggio macchina

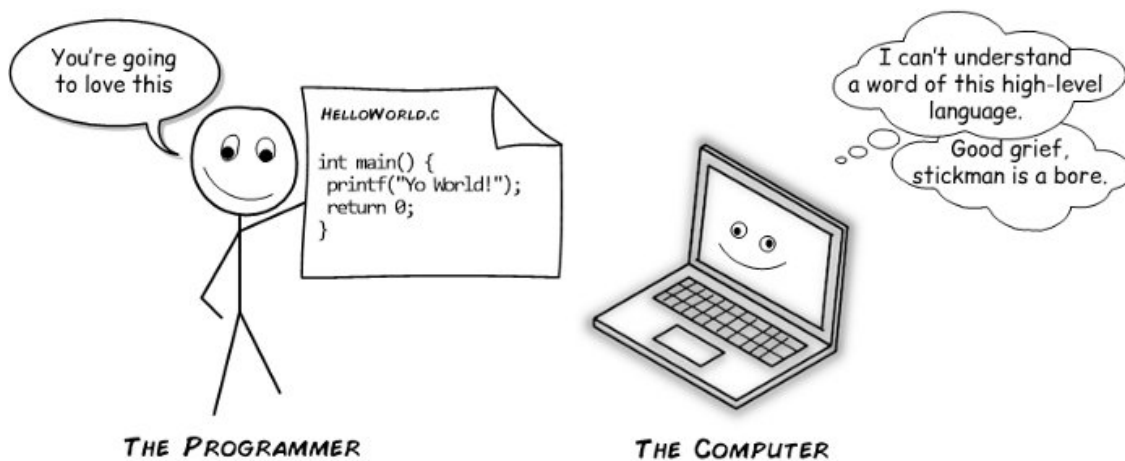


**UNIVERSITÀ
DI PARMA**

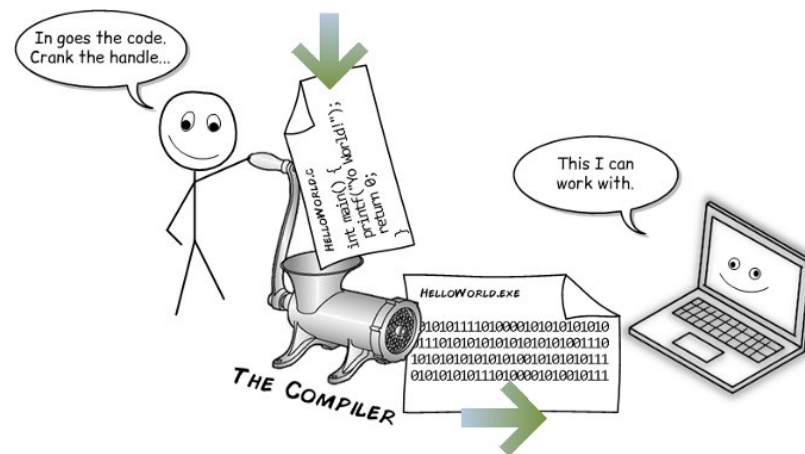
-
- A collage of various photographs of men. At the top left, three men are seated on a stage for a panel discussion. Below this, on the left, is a close-up of a man with curly hair. In the center is a portrait of a man with glasses and a beard, wearing a red shirt. To the right of the center is a portrait of a man with glasses and a beard, wearing a suit and tie. Above the center portrait is a close-up of a man's forehead and hair. To the right of the top panel is a black and white photo of a smiling man in a light-colored jacket. Below the top panel, on the right, is a close-up of a man's face with glasses. At the bottom left is a black and white portrait of a man in a suit and bow tie.

- Pro
 - Mascherano il calcolatore
 - Maggiore leggibilità e comprensibilità
 - Portabilità
- Contro
 - Necessità di traduzione

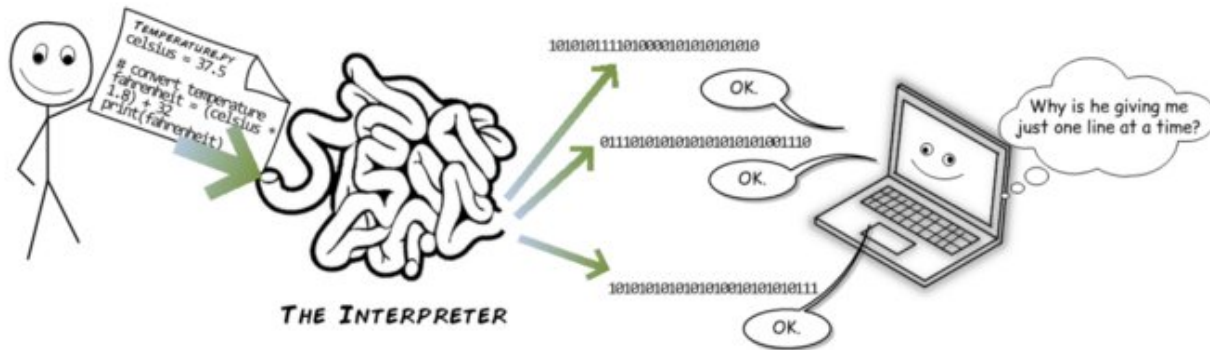
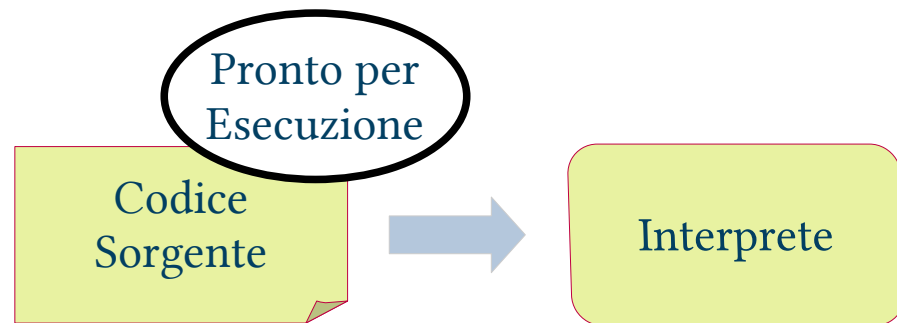
- I linguaggi ad alto livello richiedono una traduzione più complessa
 - Da codice sorgente a linguaggio macchina
- Due possibilità
 - Compilazione
 - Interpretazione
 - *Misto*



- Traduzione in codice macchina necessaria
- Compilatore → il “traduttore”
- Produce un “file eseguibile”



- Il codice sorgente non deve essere tradotto
- Interprete che lo “legge” al volo
 - Macchina virtuale



- Compilazione
 - Più veloce
 - Permette progetti multifile con più linguaggi
 - Posso nascondere codice sorgente
- Interpretazione
 - Esecuzione immediata
 - Scripting
 - Multiplatforma
 - Web e altro
- Situazioni intermedie
 - Esempio: Bytecode + JVM (JAVA)

Perché così tanti linguaggi?

- Ricerca costante di sistemi migliori per aggiornamento tecnologia
- Non solo “General Purpose” ma problemi specifici
 - Web → Javascript, PHP, GO...
 - Mobile → JAVA
 - Firmware → C
 - Sistemi real time → C++
- Performance vs Rapidità di sviluppo
 - GO/C ↔ Javascript/Python

Perché così tanti linguaggi?

Language	Year	Paradigm	Typical use
FORTRAN	1954	Imperative	Scientific computing / numerical simulation
LISP	1958	Functional	Artificial intelligence / symbolic computation
ALGOL	1958	Imperative	Academic / basis for most modern languages
COBOL	1959	Imperative	Business data processing / banking
Pascal	1970	Imperative	Education / teaching structured programming
C	1972	Imperative	OS kernels / embedded / firmware / systems
Prolog	1972	Logic	AI / expert systems / constraint solving
C++	1985	Imperative + OO	Game engines / real-time systems / HPC
Haskell	1990	Functional	Compilers / finance / research
Python	1991	Imperative + OO	Scripting / data science / AI / web
Java	1995	OO	Mobile (Android) / enterprise / web backend
JavaScript	1995	Imperative + OO	Web frontend / web backend (Node.js)
PHP	1995	Imperative + OO	Web backend / CMS
Ruby	1995	OO	Web backend (Rails) / scripting
Go	2009	Imperative	Web services / cloud infrastructure / CLI tools
Rust	2015	Imperative	Systems programming / WebAssembly / embedded

- Lessico
 - Parole chiave: istruzioni, operatori, simboli...
- Sintassi
 - Come si possono combinare i precedenti elementi
 - Parsing
 - Si può distinguere tra
 - Sintassi necessaria e
 - *Zuccherò Sintattico*
- Semantica
 - Significato di quanto scritto

**Errore di
Compilazione o
Interpretazione**

**Errore di
Esecuzione**

- Computer: sistema che esegue una serie di istruzioni in “sequenza” → automa
- Programmare:
 - Individuare algoritmo risolutivo del problema dato
 - Passi e loro ordine di esecuzione
 - Tradurre l'algoritmo individuato in codice
 - Lessico, Sintassi e Semantica specifici del linguaggio di programmazione



UNIVERSITÀ DI PARMA

I Linguaggi di Programmazione



KEEP
CALM
IT'S
QUESTION
TIME

*There will always be things we wish to say in
our programs that in all languages can only
be said poorly*

Alan Perlis